

Inception & ResNet

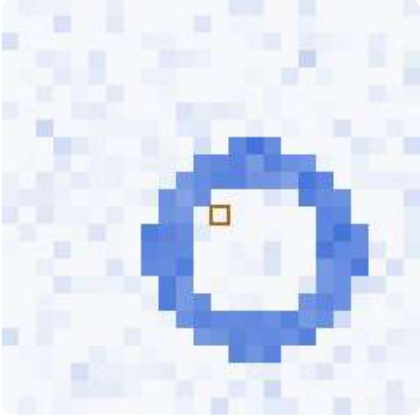
Investigate the limits of a plain convolutional network. Build a solution one decision at a time, observe its effect, and assemble the complete architecture.

Receptive Fields and Filter Size

Can one value tell us whether it belongs to a corner, a line, or a larger outline?

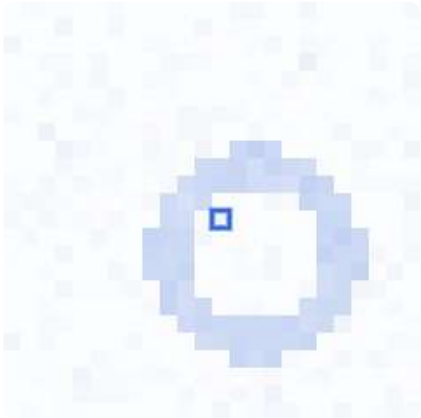
A convolution combines values within its local window. Compare three windows at the same position, then click a window to inspect the values inside it.

Input feature map



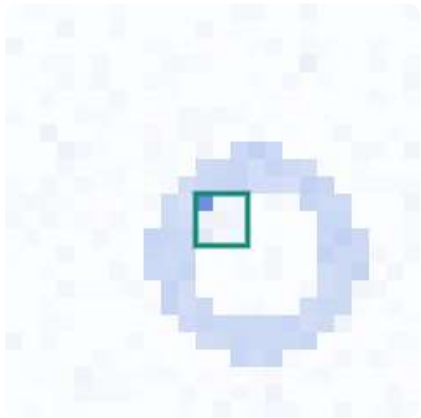
Same input for every window

1 × 1 window



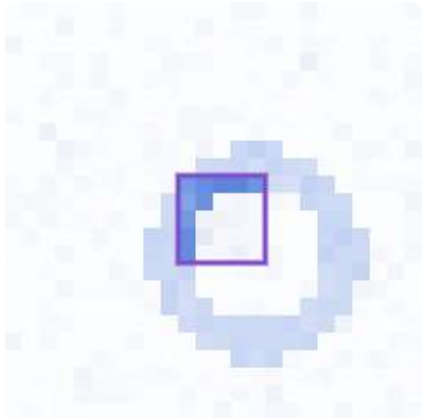
Click to magnify

3 × 3 window



Click to magnify

5 × 5 window



Click to magnify

INSIDE THE SELECTED WINDOW

3 × 3 neighbourhood · 9 values

Each enlarged cell corresponds to one highlighted input value. Move the window to compare different parts of the pattern.

0.72	0.00	0.01
0.03	0.03	0.02
0.10	0.00	0.00

Why retain 1 × 1? It mixes information across channels at each position without combining neighbouring positions. It can also change the number of channels at relatively low cost.

CHECK YOUR UNDERSTANDING

What changes when a filter grows from 3 × 3 to 5 × 5?

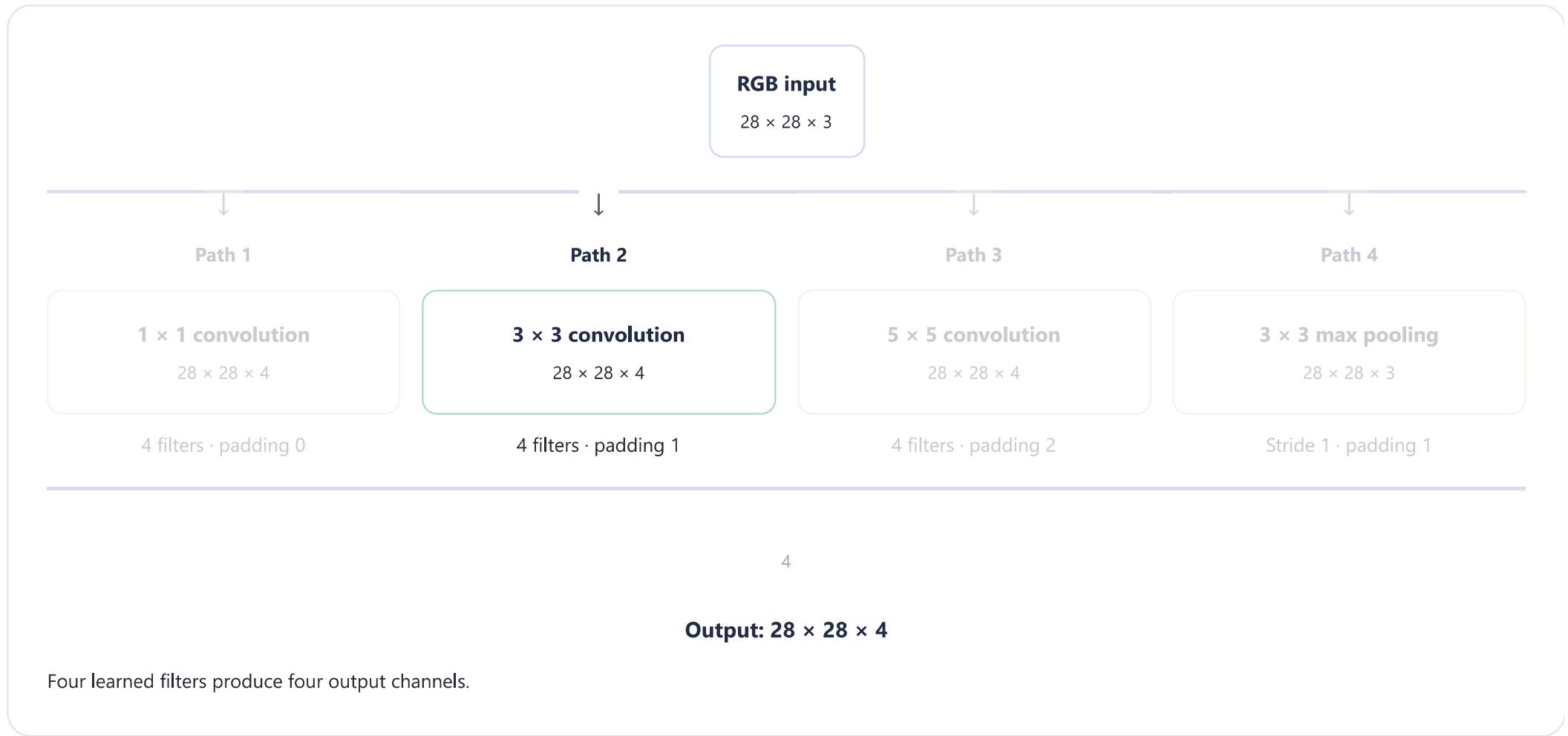
- A. It can use a wider spatial neighbourhood.
- B. It must produce more output channels.
- C. Its output must always be larger.

A single filter size provides one local window size. Parallel paths allow a layer to use several sizes together.

Combining Multiple Feature Paths

What if a layer could use a narrow view, a wider view, and a local maximum together?

Begin with a 28×28 colour image: its three channels are red, green and blue. Send the same input into parallel branches, then concatenate their output channels.



Why a parallel pooling branch? It supplies local maximum responses alongside the learned convolutional responses. Here, stride 1 and padding preserve the grid, so pooling provides another feature path rather than downsampling the whole network.

CHECK YOUR UNDERSTANDING

Three convolution branches produce four channels each. A pooling branch retains three channels. How many channels are concatenated?

- A. 4
- B. 15
- C. 12

► Inspect learned responses at different object sizes

The branches supply different local operations. As earlier layers produce more channels, applying large filters to every channel becomes expensive.

Reducing the Cost of Large Filters

A layer receives 64 feature channels. Must every 5×5 filter process all 64 directly?

An earlier convolution has produced a $28 \times 28 \times 64$ feature tensor. We want 32 output channels. Compare two ways to produce that same output shape.

STEP 1 · ESTABLISH THE COST

Direct 5×5 Convolution



One filter contains 64 spatial slices.
Each slice has 25 weights. One bias is added after the contributions from all input channels are combined.

Weights in one filter	$5 \times 5 \times 64 = 1,600$
Include its bias	$1,600 + 1 = 1,601$
Across 32 filters	$1,601 \times 32 = \mathbf{51,232}$

STEP 2 · CHANGE WHAT THE LARGE FILTER RECEIVES

Channel Reduction with 1×1 Filters

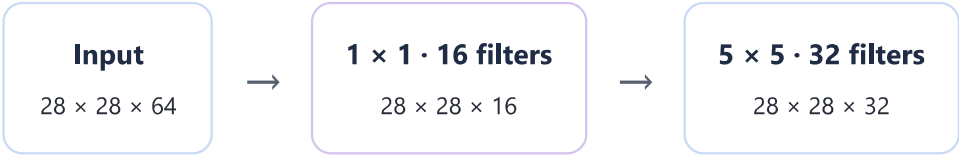
A 1×1 filter reads all channels at one position and makes one weighted combination. Using 16 such filters converts 64 input channels into 16 output channels while preserving the 28×28 grid.



► Inspect one small channel-mixing example

STEP 3 · CALCULATE BOTH LAYERS

1 × 1 Reduction Followed by 5 × 5 Convolution



Reduction layer

$$(1 \times 1 \times 64 + 1) \times 16 = 1,040$$

Large-filter layer

$$(5 \times 5 \times 16 + 1) \times 32 = 12,832$$

Both layers together

$$1,040 + 12,832 = \mathbf{13,872}$$

Same output dimensions; 51,232 → 13,872 parameters. The large filters now process 16 channels instead of 64. The reduction layer adds some work, but saves much more work in the large-filter layer.

The reduction width is a design choice: fewer channels reduce cost, but also restrict how much information can pass through.

Calculate the direct path

64 → 32 channels

+

Calculate the reduced path

64 → 16 → 32 channels

+

CHECK YOUR UNDERSTANDING

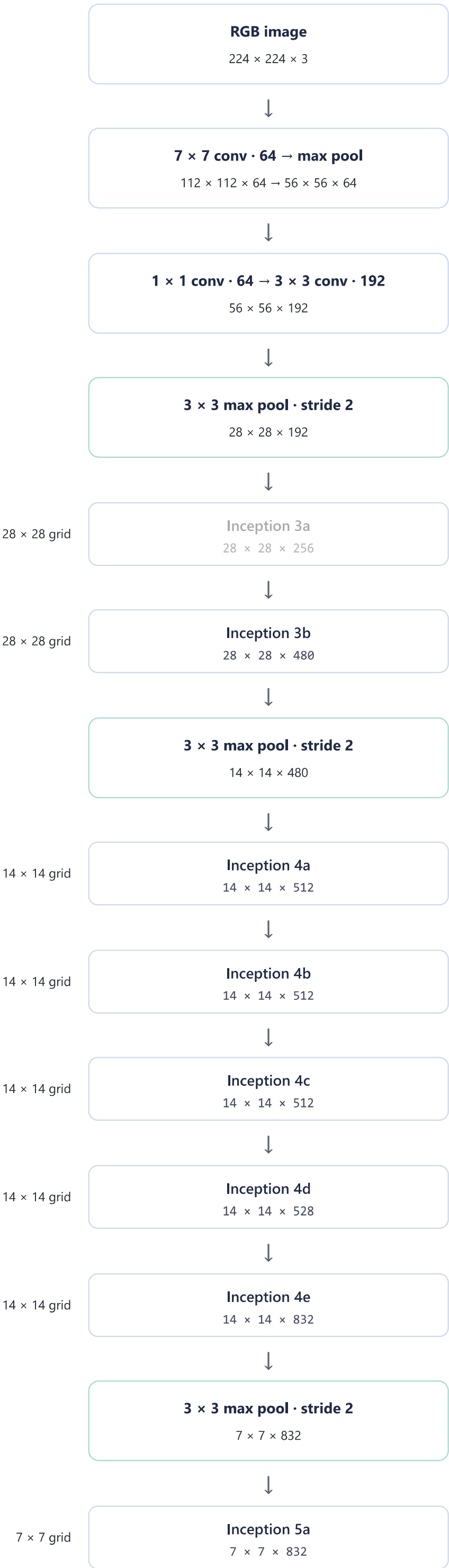
Why is the 5 × 5 convolution cheaper after the 1 × 1 reduction?

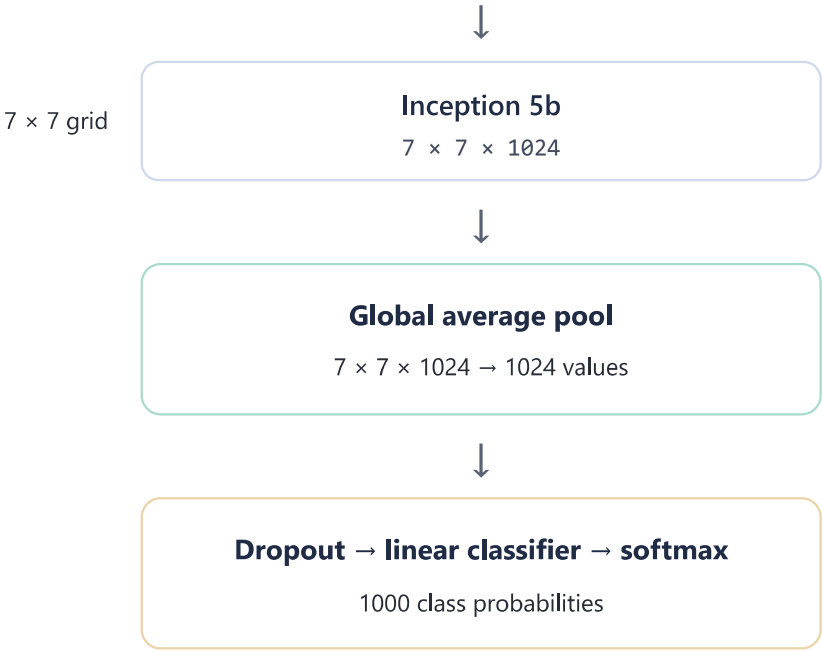
- A. Its spatial output becomes smaller.
- B. Each filter now spans 16 input channels instead of 64.
- C. Its weights are no longer learned.

Place these reductions before the large filters in a parallel module. The network can then repeat the module without an uncontrolled increase in computation.

GoogLeNet: Connected Modules and Classifiers

Follow the complete main path from top to bottom. The nine Inception modules form a deep network; select any module to inspect its parallel branches below.





Inception 3a · four aligned paths



Every convolution is followed by ReLU. Within a module, stride 1 and padding keep each branch at the same spatial size.

From Final Feature Maps to a Class Prediction

Global average pooling turns each final feature map into one value. A single linear classifier combines those values into class scores, and softmax produces class probabilities.



The large stack of fully connected layers is replaced by spatial averaging and one final linear layer. Dropout is applied before the linear layer during training.

Input photograph



224 × 224 RGB input

Feature map 6



7 × 7 activations · 49 values

Predicted classes

Samoyed	92.7%
Pomeranian	3.3%
keeshond	1.7%
white wolf	0.4%
Eskimo dog	0.4%

Top five of 1000 classes

$$z_6 = \frac{\text{Sum of 49 activations}}{49} = \frac{339.586}{49} = \mathbf{6.930}$$

► Inspect the 49 activations and the class-score calculation

CHECK YOUR UNDERSTANDING

What reaches GoogLeNet’s final linear classifier after global average pooling?

- A. 49 spatial positions per channel.
- B. One mean value from each of the 1024 channels.
- C. A class label from every channel.

Preserving a Useful Representation

More parameters provide more possible transformations. The next experiment asks how easily training can turn those possibilities into useful predictions.

Depth and Training Performance

How can a shallow network learn better when a deeper network has more parameters?

Train two networks to recognize the same handwritten digits. Compare their training loss first, then inspect their predictions on the same held-out images.

Digit Classification

Shallow plain · 4 layers

28 × 28 input

↓ 3 × 3 stem · 8 channels + pool

↓ [3 × 3 → ReLU → 3 × 3 → ReLU] × 1

↓ Flatten 14 × 14 × 8

↓ Linear classifier → 10 classes

16,938 parameters

Deep plain · 26 layers

28 × 28 input

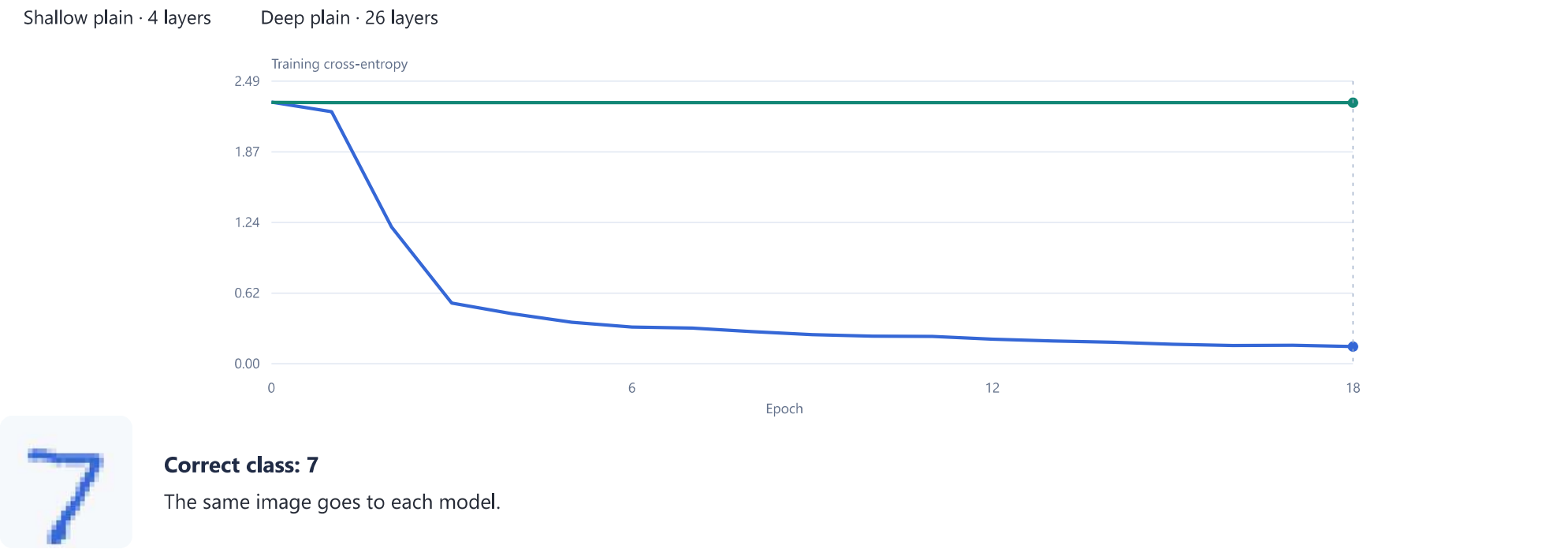
↓ 3 × 3 stem · 8 channels + pool

↓ [3 × 3 → ReLU → 3 × 3 → ReLU] × 12

↓ Flatten 14 × 14 × 8

↓ Linear classifier → 10 classes

29,786 parameters



Shallow plain · 4 layers			Deep plain · 26 layers		
0.1503			2.2998		
Training loss			Training loss		
Prediction: 7 · 100.0%			Prediction: 7 · 11.1%		
0	0.0%		0	9.4%	
1	0.0%		1	10.9%	
2	0.0%		2	9.9%	
3	0.0%		3	9.7%	
4	0.0%		4	11.0%	
5	0.0%		5	9.3%	
6	0.0%		6	10.0%	
7 · correct class	100.0%		7 · correct class	11.1%	
8	0.0%		8	8.8%	
9	0.0%		9	10.0%	

The deeper plain network has 29,786 parameters but ends with training loss 2.300. The shallow network has 16,938 parameters and reaches 0.150. Extra capacity alone has not solved the training problem.

Recorded training on 2400 images; test measurements use 600 separate images. The slider replays measured epochs. Predictions below refer to the selected test image.

Capacity and successful training are different. More layers add possible transformations, but also add transformations that must learn to work together. An early layer's update depends on the layers after it, which are also changing during training. A shorter chain can reach a useful solution more easily.

Higher training loss means the deeper network has not fitted the training examples as well. This is an optimization difficulty. It is not explained by overfitting alone.

CHECK YOUR UNDERSTANDING

The deeper plain network has more parameters but higher training loss. What has the comparison shown?

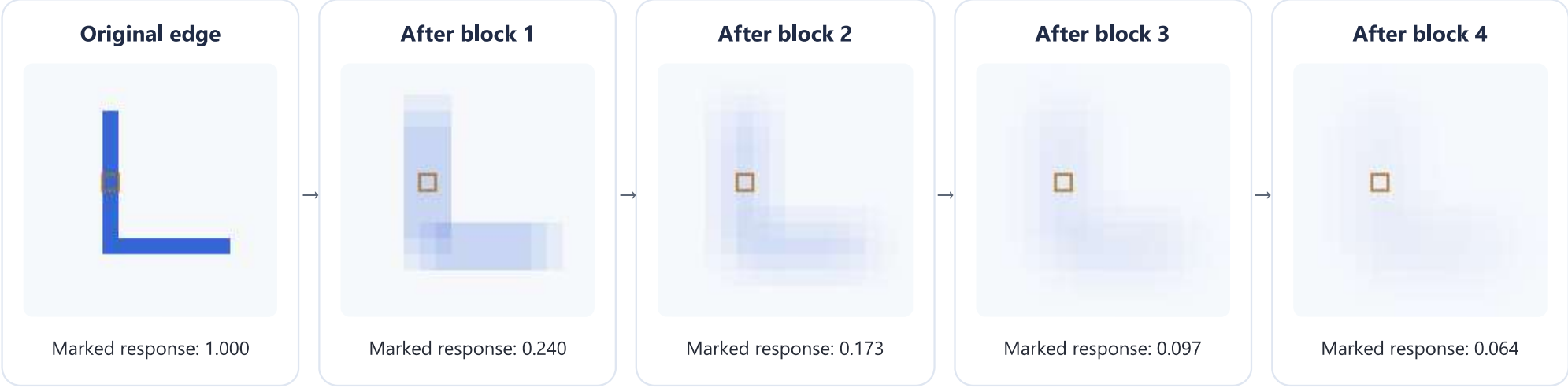
- A. Additional capacity has not been successfully used by training.
- B. Fewer parameters are always better.
- C. The deeper model has necessarily overfit.

Preserving a Feature Through Additional Layers

An early convolution has found a useful edge. Can later blocks preserve it while learning what to change?

Follow One Edge Response

Apply successive transformations to the same edge map. Keep the original visible while inspecting the output of each step.

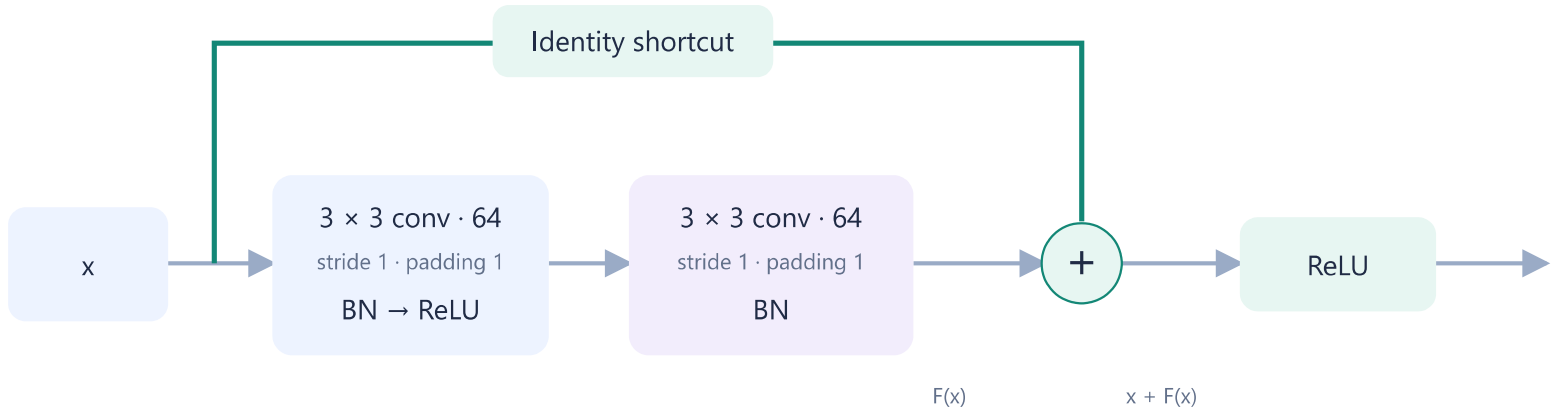


After 4 transformations, the marked response changes from 1.000 to 0.064. The learned path must coordinate its transformations to produce a useful final representation.

This controlled example uses local averaging and scaling, making changes to a response easy to inspect. A changed feature is not automatically a worse feature; the training curves establish the performance difference.

In the plain path, all information must pass through the learned transformations. Introduce a direct path around a block so the input can also reach its output without being transformed by that block.

Input Plus a Correction



$y = x + F(x)$

The correction is zero. The input still reaches the addition unchanged through the direct path.

Set the correction to inspect the addition. Positive corrections increase responses; negative corrections decrease them. The maps share a fixed colour scale.

A plain block must learn the complete output. A residual block can preserve its input directly and learn how to modify it. At the addition, a zero residual gives the input back. The following ReLU preserves nonnegative input features.

► How the shortcut also helps the backward path

CHECK YOUR UNDERSTANDING

If the residual $F(x)$ is zero, what reaches the addition output?

- A. Zero.
- B. The input x .
- C. Twice the input.

Training with Residual Connections

Now add shortcuts to the deep network. Its learned layers, parameter count, initialization and training batches match the deep plain network. Compare the complete training runs.

Digit Classification

Shallow plain · 4 layers

- 28 × 28 input
- ↓ 3 × 3 stem · 8 channels + pool
- ↓ [3 × 3 → ReLU → 3 × 3 → ReLU] × 1
- ↓ Flatten 14 × 14 × 8
- ↓ Linear classifier → 10 classes
- 16,938 parameters

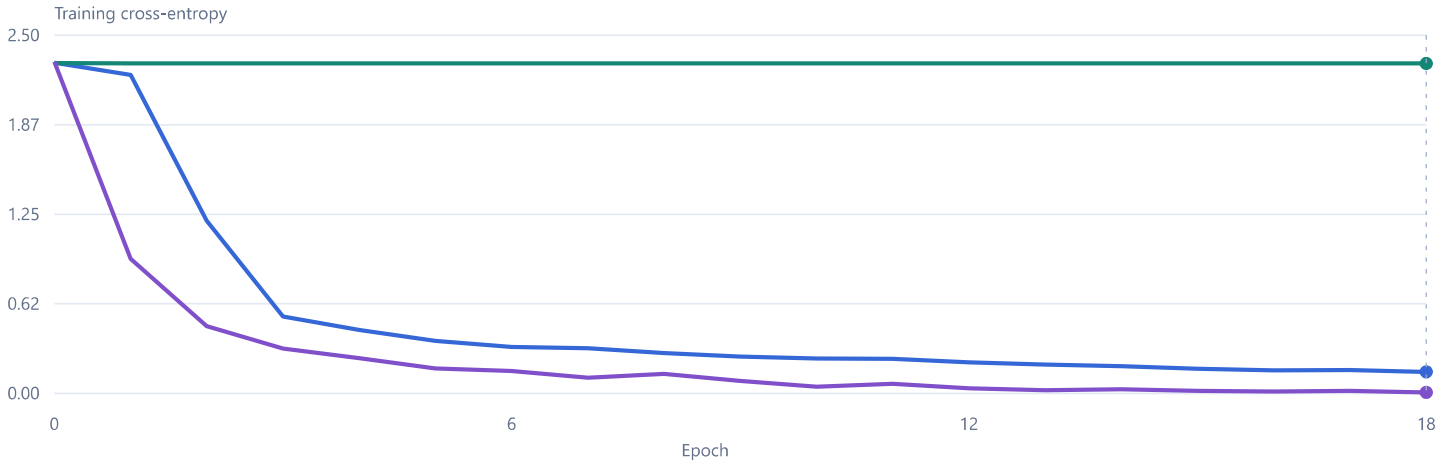
Deep plain · 26 layers

- 28 × 28 input
- ↓ 3 × 3 stem · 8 channels + pool
- ↓ [3 × 3 → ReLU → 3 × 3 → ReLU] × 12
- ↓ Flatten 14 × 14 × 8
- ↓ Linear classifier → 10 classes
- 29,786 parameters

Deep residual · 26 layers

- 28 × 28 input
- ↓ 3 × 3 stem · 8 channels + pool
- ↓ [3 × 3 → ReLU → 3 × 3 + shortcut → ReLU] × 12
- ↓ Flatten 14 × 14 × 8
- ↓ Linear classifier → 10 classes
- 29,786 parameters

Shallow plain · 4 layers Deep plain · 26 layers Deep residual · 26 layers



Correct class: 7
The same image goes to each model.

Shallow plain · 4 layers

0.1503
Training loss

Prediction: 7 · 100.0%

0	0.0%
1	0.0%
2	0.0%
3	0.0%
4	0.0%
5	0.0%
6	0.0%
7 · correct class	100.0%
8	0.0%
9	0.0%

Deep plain · 26 layers

2.2998
Training loss

Prediction: 7 · 11.1%

0	9.4%
1	10.9%
2	9.9%
3	9.7%
4	11.0%
5	9.3%
6	10.0%
7 · correct class	11.1%
8	8.8%
9	10.0%

Deep residual · 26 layers

0.0068
Training loss

Prediction: 7 · 100.0%

0	0.0%
1	0.0%
2	0.0%
3	0.0%
4	0.0%
5	0.0%
6	0.0%
7 · correct class	100.0%
8	0.0%
9	0.0%

The deep residual model reaches training loss 0.0068 and test accuracy 95.3%. Its plain counterpart has the same 29,786 parameters. The difference is how the blocks use their input.

Recorded training on 2400 images; test measurements use 600 separate images. The slider replays measured epochs. Predictions below refer to the selected test image.

► Experimental setup and measured residual maps

CHECK YOUR UNDERSTANDING

What changes between the deep plain and deep residual digit networks?

- A. The training examples.
- B. The number of trainable parameters.
- C. Identity additions around the two-convolution blocks.

The shortcut provides a useful way to express the computation. The measured loss and predictions show what this change achieves during training.

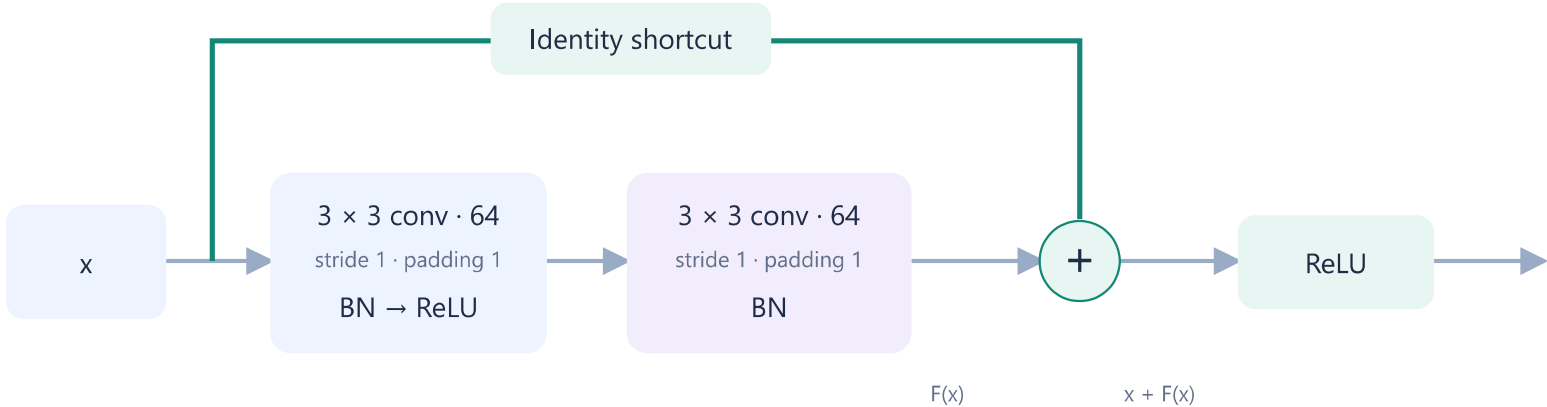
ResNet-18: Connected Residual Stages

Follow the main path through four stages. Each stage contains two residual blocks. Select a block to inspect both its main path and its shortcut.



Stage 1 · Block 1

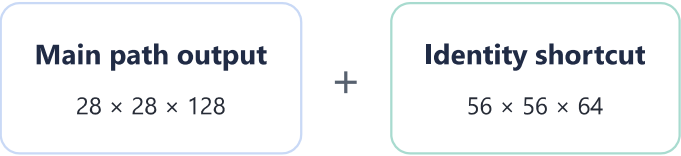
Input: $56 \times 56 \times 64 \rightarrow$ output: $56 \times 56 \times 64$. The shortcut preserves the input without learned weights.



BN denotes batch normalization, which normalizes and rescales channels. The main path has one stem convolution, sixteen block convolutions and one final linear layer: eighteen learned layers.

Matching Shapes at a Stage Boundary

The main path halves the image size and doubles the channels. Can its output still be added to the original input?



The shapes do not match. A 28 × 28 × 128 tensor cannot be added element by element to a 56 × 56 × 64 tensor.

A learned 1 × 1 projection sets the shortcut to 128 channels. Stride 2 halves its spatial size. Batch normalization follows the projection.

Trace the projection shortcut

1 × 1 · stride 2 · padding 0

+

CHECK YOUR UNDERSTANDING

Which shortcut matches 56 × 56 × 64 to 28 × 28 × 128?

- A. Identity only.
- B. 1 × 1 convolution with 128 filters and stride 2.
- C. Max pooling with stride 1.

Two Designs, Two Merge Operations

Inception

Extract features through parallel paths. Use channel reduction to control cost. **Concatenate** the channel groups.

0 of 9 checks answered correctly · 0 attempted

Reset MCQ answers

ResNet

Retain a direct path for the input. Learn a residual change. **Add** corresponding values after matching the shapes.

CHECK YOUR UNDERSTANDING

Which statement correctly distinguishes the two merge operations?

- A. Inception concatenates channel groups; ResNet adds matched values.
- B. Both operations always double the channels.
- C. Both operations average the branch outputs.

Deep Learning · Worksheet 11 · All diagrams, recorded training and feature maps are included in this file. No runtime downloads are required.

Further reading: [Going Deeper with Convolutions](#) [Deep Residual Learning for Image Recognition](#)

[← Previous: LeNet, AlexNet & VGG-16](#)